

UJIAN TENGAH SEMESTER (UTS)

Take-Home Individu: Mini Dashboard Full-Stack (Vue 3 + FastAPI)

Web Application Development

1. Pendahuluan

UTS ini berbentuk take-home individu. Setiap mahasiswa membangun satu mini dashboard full-stack (Vue 3 di sisi frontend, FastAPI di sisi backend) dari nol, sebagai gabungan dari seluruh materi Pertemuan 1 sampai Pertemuan 7: arsitektur client-server, konsumsi API asinkron, Composition API Vue, routing dan skema Pydantic di FastAPI, hingga kontrak REST/OpenAPI.

2. Ketentuan Umum

- Dikerjakan secara individu di perangkat masing-masing, selama waktu yang ditentukan.
- Nomor urut/NIM GANJIL mengerjakan Soal A (Dashboard Perpustakaan); GENAP mengerjakan Soal B (Dashboard Inventaris).
- Data awal (seed) **wajib dibuat sendiri oleh mahasiswa**. Hal ini supaya tampilan dashboard tiap mahasiswa berisi data yang berbeda-beda dan tidak bisa disalin utuh dari mahasiswa lain.
- Histori commit menjadi bagian bukti kewajaran pengerjaan dan termasuk salah satu komponen penilaian.
- Sertakan file README.md singkat: cara menjalankan backend dan frontend, serta penjelasan ambang batas (threshold) status stok yang dipakai
- Submit link repository GitHub melalui media yang telah ditentukan.
- Dashboard yang dibuat wajib dipresentasikan secara langsung sebagai bagian dari penilaian UTS di pertemuan 8.

3. Spesifikasi Teknis

Spesifikasi berikut wajib dipenuhi baik oleh Soal A (Perpustakaan) maupun Soal B (Inventaris) antara lain sebagai berikut:

3.1 Vue 3 (Composition API)

- Data dari backend disimpan sebagai `ref`
- Pencarian/filter menggunakan `computed` (berdasarkan salah satu field teks utama)
- Tombol urutkan A-Z / Z-A menggunakan `computed` berantai di atas hasil pencarian
- Badge/label status stok memakai conditional class binding, berubah otomatis sesuai data yang diterima dari backend
- Form tambah data baru memakai state lokal (`ref`), mengirim POST ke backend, lalu me-refresh daftar
- Tombol hapus per item memanggil DELETE ke backend dan memperbarui tampilan
- Baris ringkasan (4 tile statistik) di bagian atas dashboard, dihitung murni dengan `computed` dari data yang sudah diambil — rinciannya ada di Bagian 4/5

3.2 JavaScript (ES6+)

- Transformasi data (filter pencarian, urutkan, hitung 4 tile ringkasan) wajib pakai method array native — `.filter()`, `.sort()`, `.map()`, `.reduce()` — **tidak boleh** pakai library utilitas eksternal (lodash dan sejenisnya)
- Pengambilan data asinkron wajib pakai `async/await` dengan `try/catch` untuk menangani kegagalan — bukan `.then()` berantai tanpa penanganan error

3.3 Fetch API & Penanganan Status

- Seluruh komunikasi ke backend memakai `fetch` bawaan browser
- Status pengambilan data **wajib** direfleksikan di UI (bukan hanya `console.log`): **loading** saat menunggu respons, **kosong** kalau hasil pencarian tidak ada, **error** kalau gagal terhubung ke backend, dan **berhasil** saat data tampil

3.4 FastAPI

- Endpoint REST standar: GET (ambil semua data), POST (tambah data baru), DELETE (hapus data berdasarkan id) — ketiganya wajib berfungsi
- Skema request dan response didefinisikan dengan Pydantic, termasuk validasi field wajib dan tipe data (contoh: stok harus angka, tidak boleh string)
- Data disimpan sebagai **list Python in-memory**, di-seed dari file JSON buatan mahasiswa sendiri saat server pertama kali dijalankan — bukan database
- CORS diaktifkan di FastAPI supaya frontend Vue lokal dapat memanggil backend lokal tanpa diblokir browser

3.5 Responsif UI (Desktop / Tablet / Mobile)

- Breakpoint minimal: **Desktop** $\geq 1024\text{px}$, **Tablet** 768-1023px, **Mobile** $< 768\text{px}$
- Kriteria dinilai secara fungsional, bukan pixel-perfect: tidak ada horizontal scroll yang tidak disengaja pada lebar 375px
- Baris tile ringkasan boleh menyusut dari 4 kolom sejajar menjadi 2 kolom atau ditumpuk penuh di layar mobile
- Toolbar (pencarian, tombol urutkan, tombol tambah) boleh membungkus ke baris baru — bukan terpotong atau tertutup elemen lain
- Tabel boleh discroll horizontal, atau berubah jadi daftar kartu, di layar sempit
- Cukup pakai CSS Flexbox/Grid dan media query native

Fitur bonus (opsional, nilai tambah)

Endpoint PUT/PATCH untuk mengubah data (misalnya update stok) beserta tombol edit di frontend, dokumentasi OpenAPI (/docs) yang dirapikan dengan deskripsi tiap endpoint, dan visualisasi "Ringkasan Stok per Kategori" berupa bar sederhana berbasis CSS (lebar bar dihitung dari computed, **bukan** library chart apa pun)

4. Soal A (Nomor Urut/NIM Ganjil) — Dashboard Perpustakaan

Entitas: Buku — field: id, judul, penulis, kategori, stok (angka).

Bangun mini dashboard yang menampilkan daftar buku perpustakaan, dapat dicari, diurutkan, ditambah, dan dihapus, sesuai spesifikasi teknis umum di Bagian 3.

Ketentuan spesifik Soal A:

- 1. Pencarian berdasarkan judul atau penulis
- 2. Urutkan A-Z / Z-A berdasarkan judul buku
- 3. Badge status stok: **Tersedia**, **Menipis**, **Stok Habis** — ambang batas (berapa angka stok termasuk "Menipis") ditentukan sendiri oleh mahasiswa, asalkan masuk akal dan dijelaskan di README
- 4. 4 tile ringkasan di atas daftar: **Total Buku** (jumlah semua buku), **Stok Menipis + Habis** (jumlah buku dengan status itu), **Jumlah Kategori** (kategori unik), dan **Total Eksemplar** (jumlah seluruh angka stok dijumlahkan) — keempatnya dihitung dengan `computed` dari data buku yang sama, tidak perlu endpoint atau entitas baru
- 5. Minimal 20 data buku buatan sendiri sebagai seed awal

5. Soal B (Nomor Urut/NIM Genap) — Dashboard Inventaris

Entitas: Barang — field: `id`, `nama`, `kategori`, `jumlah_stok` (angka), `lokasi_gudang`.

Bangun mini dashboard yang menampilkan daftar barang inventaris, dapat dicari, diurutkan, ditambah, dan dihapus, sesuai spesifikasi teknis umum di Bagian 3.

Ketentuan spesifik Soal B:

- 1. Pencarian berdasarkan nama atau kategori barang
- 2. Urutkan A-Z / Z-A berdasarkan nama barang
- 3. Badge status stok: **Aman**, **Menipis**, **Habis** — ambang batas ditentukan sendiri oleh mahasiswa, asalkan masuk akal dan dijelaskan di README
- 4. 4 tile ringkasan di atas daftar: **Total Barang** (jumlah semua barang), **Stok Menipis + Habis** (jumlah barang dengan status itu), **Jumlah Kategori** (kategori unik), dan **Total Unit** (jumlah seluruh angka stok dijumlahkan) — keempatnya dihitung dengan `computed` dari data barang yang sama, tidak perlu endpoint atau entitas baru
- 5. Minimal 20 data barang buatan sendiri sebagai seed awal

6. Bobot Penilaian

Kriteria berikut berlaku sama untuk Soal A maupun Soal B.

Komponen Penilaian	Poin
Backend (FastAPI) — endpoint GET/POST/DELETE berfungsi & validasi Pydantic benar	20
Frontend (Vue) — daftar, pencarian, urutkan (<code>computed</code> berantai), dan 4 tile ringkasan berjalan benar	20
Fetch API & penanganan status (loading/kosong/error/berhasil tampil di UI)	10
Responsivitas UI di 3 breakpoint (desktop/tablet/mobile)	10
Integrasi frontend-backend — CORS aktif, tambah & hapus data benar-benar mengubah data di backend (bukan hanya tampilan lokal)	15
Data seed buatan sendiri (unik, minimal 20 data, sesuai skema) & kerapian README	10

Komponen Penilaian	Poin
Kualitas kode (termasuk penggunaan method array JS native & async/await) & history commit bertahap	5
Fitur bonus (update/edit data, dokumentasi OpenAPI rapi, visualisasi CSS opsional)	5
Presentasi/demo singkat pada sesi Pertemuan 8	5
Total	100